

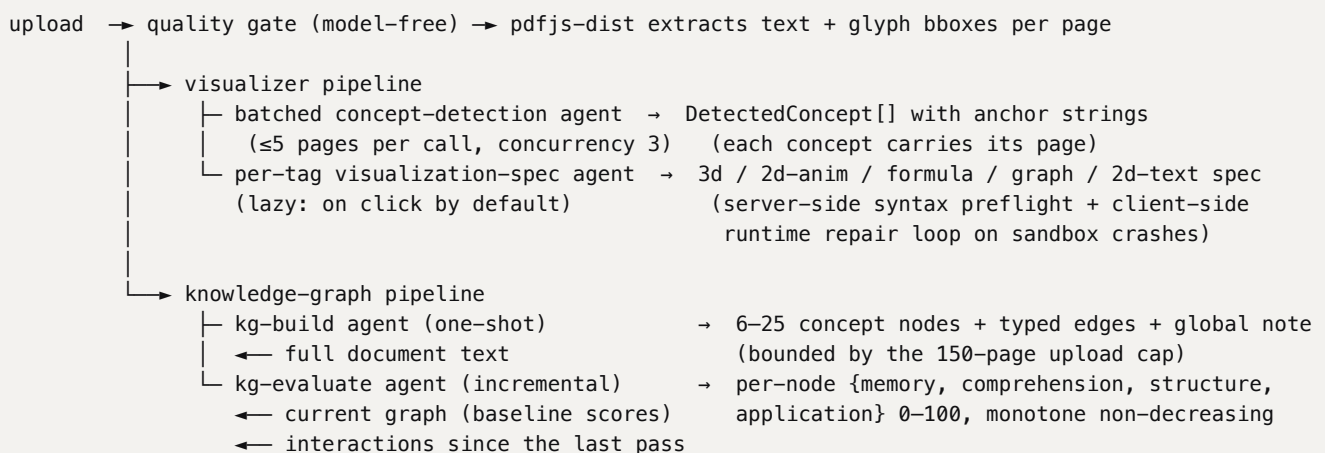
Get It. • Technical Writeup

"What I cannot create, I do not understand." • Richard Feynman, last blackboard at Caltech, February 15, 1988.

A developer-oriented walk through the codebase. The goal is that a contributor who has never read the project before can finish this document and confidently navigate the source, change a piece of behaviour, or replace a layer.

Mental model

Get It. is a desktop app for studying a single PDF at a time. Drop a file — a fast, model-free quality gate first confirms the PDF is a digital, text-based document under 150 pages; a scanned, image-dominant, or text-sparse file is rejected up front with a clear message instead of silently feeding empty text to the agents. Once a file clears the gate, two pipelines fire in parallel from upload. The first one ships visualizations inline next to the text so the document is immediately easier to read. The second one builds a concept graph of the same document and scores the student's mastery on four orthogonal axes as they interact with four study tools (chat, flashcards, forced-choice quizzes, Feynman with a curious child). Every interaction lands in one append-only journal on disk. One evaluator agent reads that journal and updates the four-axis scores after every completed session.



Persistent state is a tree of plain JSON files under one OS-native user-data directory. There is no database, no hosted backend, and no shared key pool. Every model call is an `@openai/codex-sdk` invocation against the end user's own ChatGPT account. The only network call beyond that model traffic is an anonymous open/update ping (a random install id + app version + OS, no document content or PII) used purely for aggregate user counts on the marketing site, and disabled entirely by `GETIT_DISABLE_ANALYTICS=1`.

Code map

The product is a single Next.js 16 application wrapped in a small Electron shell.

electron/ main.js setup.js updater.js analytics.js update-window/ wizard/ codex-bin/<triple>/ preload*.js	desktop shell + setup wizard + auto-update single-instance lock, data dir resolution, server spawn Codex CLI install + OAuth wizard window GitHub Releases poll, in-app installer flow anonymous open/update ping (opt-out via env) wizard HTML/JS for the update modal wizard HTML/JS for the first-launch wizard bundled Codex CLI per platform/arch context-isolated preload bridges
app/ page.tsx library/ viewer/[docId]/ api/ upload/ analyze-pdf/ tags/[docId]/ jobs/detect/[docId] jobs/viz/[docId] chat/[docId] flashcards/[docId] quizzes/[docId] feynman/[docId] kg/[docId]/ work-context/[docId] codex/health codex/account logout	Next.js App Router pages + API routes upload home catalog of every opened PDF per-document viewer (PDF + right pane) pdfjs extraction + quality gate + new docId or sample reuse legacy single-shot detection (preserved for tests) server-owned tag store: GET / POST active-tag / etc. POST → kicks the batched concept-detection job for a doc POST → kicks per-tag viz-spec generation POST → chat turn on a native Codex thread (start / resume) POST generate / rate / end (triggers scheduleEvaluation) POST generate / answer / end (triggers scheduleEvaluation) POST start / explain (triggers scheduleEvaluation) build / state / evaluate download the journal as JSON process-local CodexError mailbox (banner polls this) surface ChatGPT account + sign-out
components/ RightPane/ Visualizer/ PdfViewer.tsx CodexHealthBanner.tsx	React UI (orchestrators + scene renderers + tag UI) mode dropdown + the four tool views + KG view 3D / 2D / formula / graph / text renderers + sandbox pdf.js viewer with overlay tag pills error banner + countdown + re-connect
lib/ codex.ts codex-errors.ts agents/ kg.ts / kg-runner.ts store.ts paths.ts pdf-extract.ts schemas.ts schemas-kg.ts work-context*.ts viz-runtime.ts config.ts	framework-agnostic helpers the one LLM transport: runJson + runJsonInThread + SDK wrapper pure error model: classifier + friendly payloads (no SDK dep) per-agent prompt builders (detect, viz) KG persistence + build/eval runners + scheduler doc cache + filesystem persistence the OS-native data-dir resolver pdfjs-dist text + bbox extraction + upload quality gate JSON schemas for every agent journal storage + evaluator summary the `new Function` sandbox compiler runtime-mutable settings + env defaults

The agent layer

Every call to OpenAI funnels through `lib/codex.ts`. The stateless workhorse is `runJson(prompt, outputSchema, opts)`; the chat tool additionally uses `runJsonInThread(...)`, which **starts or resumes a native Codex thread** so a multi-turn conversation transmits the document once and each follow-up turn carries only the new message. Both paths share the same client, sandbox, schema enforcement, and error handling. The helper:

1. Lazily initialises one `Codex` client per process.
2. Starts a fresh thread — or resumes a stored `threadId` — with an **explicitly pinned model** (`gpt-5.5`), `sandboxMode: "read-only"`, `approvalPolicy: "never"`, `skipGitRepoCheck: true`, and an explicit working directory under `<DATA_DIR>/codex-scratch`. Pinning the model matters: the SDK only passes `--model` when we set it, so if we leave it blank the bundled binary resolves the model itself — from the user's personal `~/codex/config.toml`, else a default compiled into the binary. That baked-in default can be a model OpenAI has since retired for ChatGPT-account auth, which 400s for everyone who doesn't already have a local

config overriding it. Pinning makes every install — across OS, arch, and environment — deterministically use the same supported model. The renderer never sees a turn that escaped its own working dir.

3. Runs the turn against the supplied JSON Schema, retries once on parse failure, and returns the typed result.
4. Catches every throw, classifies it into `auth_lost` / `rate_limit` / `binary_missing` / `model_unsupported` / `generic`, and writes the result into a process-local **health mailbox**. The renderer polls `/api/codex/health` to render a banner. `model_unsupported` (the pinned model aged out server-side) tells the user to update the app. Rate-limit retry deadlines are extracted from the error message when present, and when a quota message carries no deadline we apply a conservative fallback cooldown so the short-circuit below always has a window to act on.
5. Short-circuits future calls while a rate-limit window is still active so a chatty UI cannot burn a hundred wasted calls.

The pure error model (the `CodexError` kinds, the classifier, and the friendly per-kind payloads the request/response routes return) lives in a separate `codex-errors.ts` with no SDK import, so it stays unit-testable in isolation; `codex.ts` re-exports it.

Nine prompts live behind that transport:

WHERE	WHAT IT RETURNS	SCHEMA
<code>lib/agents/detect.ts</code>	<code>DetectedConcept[]</code> for a batch of up to 5 pages (each concept tagged with its page)	<code>detectionSchema</code> in <code>lib/schemas.ts</code>
<code>lib/agents/viz.ts</code>	Per-tag visualization spec (one of five renderer types)	<code>vizSchemaFor(type)</code> in <code>lib/schemas.ts</code>
<code>lib/kg-runner.ts</code> → <code>BUILD_SYSTEM</code>	The graph: 6–25 nodes, typed edges, global note	<code>kgBuildSchema</code> in <code>lib/schemas-kg.ts</code>
<code>lib/kg-runner.ts</code> → <code>EVALUATE_SYSTEM</code>	Per-node updates {memory, comprehension, structure, application} + notes	<code>kgEvaluateSchema</code>
<code>app/api/chat/[docId]/route.ts</code>	One assistant reply	<code>chatReplySchema</code>
<code>app/api/flashcards/[docId]/route.ts</code>	4–10 Q / A cards	<code>flashcardsGenerateSchema</code>
<code>app/api/quizzes/[docId]/route.ts</code>	4–8 MCQs with one correct option and three distractors	<code>quizGenerateSchema</code>
<code>app/api/feynman/[docId]/route.ts</code> → <code>CHILD_SYSTEM</code>	One curious-child prompt	<code>feynmanChildPromptSchema</code>
<code>app/api/feynman/[docId]/route.ts</code> → <code>SUMMARY_SYSTEM</code>	End-of-session honest summary	<code>feynmanSummarySchema</code>

There is no god-prompt and no client-side JSON-Schema validation. Every agent reply arrives as a typed TypeScript object the rest of the code can use without defensive parsing.

The visualizer pipeline

The pipeline that ships time-to-value: tags appear inline the instant detection returns. By default a tag's visualization is generated **lazily** — the first time the student clicks the tag — so model usage stays proportional to what they actually open; this matters on long documents, where a single PDF can carry hundreds of tags. An opt-in `auto-generate` setting flips this back to eager mode, rendering every tag in parallel as soon as detection finds it. Either way, a ready tag is marked with a thin emerald ring so the student can tell at a glance which visualizations already exist.

Server-side jobs. Detection and per-tag viz generation are not renderer loops. Both are first-class **server-side jobs**, singleton-per-doc, idempotent, running inside the Next process. The detection job walks unanalysed pages in batches of up to five pages per Codex call, runs those batches at concurrency 3, and persists each batch of new tags to `<DATA_DIR>/docs/<docId>/tags.json` as it goes — each detected concept carries the page it belongs to, so one call can tag five pages at once and a 100-page document costs roughly twenty detection calls instead of a hundred. The viz job picks the next tag whose state is `generating: true`, runs the per-type agent at concurrency 4, and persists the spec back to the same file. The viewer is a *consumer*: it polls `GET /api/tags/<docId>` every 1.5 s while any job is in flight, fires `POST /api/jobs/viz/<docId>` on a user click or a sandbox runtime-error

report, and only ever updates the *active-tag selection* on the server. The active selection is the lone field the client can write; everything else is server-owned, so a concurrent client navigation cannot overwrite mid-flight detection or generation.

Reopening a doc from the Library weeks later therefore restores the exact tag layout, viz specs, and analysed-pages set without re-detection. Library badges poll the same source so they stay live across the whole catalog with no extra plumbing.

Five renderer types. `lib/agents/viz.ts` routes by `VizType` :

- **3d** . The agent emits a JavaScript function body that `components/Visualizer/ThreeDView.tsx` executes with `{ THREE, scene, camera, renderer, controls, group }` in scope. The viewer auto-frames the molecule with a bbox and auto-rotates the group.
- **2d-anim** . Same shape, but the function body returns an object with `draw(ctx, width, height, time, dt)` and runs every frame on a Canvas2D context.
- **formula** . A headline LaTeX line plus 2–6 derivation steps with one-sentence explanations; rendered with KaTeX.
- **graph** . A `chart_type` (function / points / bars / lines) plus a JSON-string `data_json` ; plotted on a Canvas.
- **2d-text** . Title plus caption plus markdown body plus citation list. Used for legal articles, named papers, and authoritative quotations. Web search is enabled only for this type.

The sandbox. `lib/viz-runtime.ts → compileFn` wraps each LLM-emitted function body in an IIFE that shadows the dangerous globals (`window`, `document`, `fetch`, `XMLHttpRequest`, `WebSocket`, `Function`, `eval`, `localStorage`, `sessionStorage`, `require`, `Worker`, `WebAssembly`, `process`, `globalThis`) as `undefined` parameters before the inner function runs. The boundary is a defense against LLM mistakes, not against adversarial input: the user is running their own Codex account against their own PDFs.

Repair loop. When the sandbox throws inside `ThreeDView`'s `setup_code` or in a `2d-anim` `draw`, the viewer reports the error string back to the server, which hands it to Codex as repair context (the broken `setup_code` + the captured error message) and asks for a corrected JSON object that compiles and runs end-to-end. The user sees "repairing, attempt N of M" instead of red text. Server-side syntax pre-flight via `new Function(...)` catches truncated bodies before they ever leave the route.

The knowledge-graph pipeline

This is the layer that turns Get It. from a viewer into a measurement instrument. Two agents, one persistence file, one queue.

kg-build runs once per document at upload time. The system prompt asks for 6–25 concept nodes the student would actually need to master (not a glossary), typed edges (prerequisite / composition / causal / specialisation / contrast), and a short global note that the viewer prints above the graph. Output is written to `<DATA_DIR>/docs/<docId>/kg.json` with `status: "ready"`. Any failure — an account-level Codex error or a one-off — moves the graph to `status: "error"` with the reason, so the KG view drops its spinner and shows a **Retry** button. There is no automatic retry: the build re-runs on demand, and the health banner explains an account-level failure in the meantime.

kg-evaluate is the four-axis rubric. Every node carries four 0–100 scores:

AXIS	WHAT IT MEASURES	STRONGEST SIGNAL
memory	recall over time	flashcard ratings (1–4), quiz correctness on definitional questions, recall references in chat
comprehension	understanding in the student's own words	original metaphors in chat, plain-language Feynman explanations, distractor-rejection in quizzes
structure	grasp of how concepts connect	multi-step reasoning that bridges concepts, references to prerequisites, sibling discrimination
application	transfer to new cases	original examples, edge cases, novel problem solving, applied-tier quiz answers

The evaluator sees the current graph with each node's previous scores as a baseline, plus **only the interactions since the last pass** (compacted via `summariseForEvaluator`, which filters the journal by timestamp). Earlier evidence is already encoded in the baseline, so a pass stays cheap no matter how long the journal grows — and

because scores only ever rise, nothing is lost by not re-reading the old transcript. Its system prompt enforces three rules: scores are **monotone non-decreasing**, *quantity does not entitle a score*, and concepts with no observable evidence stay at their previous level. The runtime enforces the monotone rule with a clamp on every update (`clampMonotone` in `lib/kg-runner.ts`) so a chatty interaction cannot accidentally erase prior evidence even if the agent disregards its own instruction.

Scheduling. Each evaluator pass is one Codex turn at medium effort, coalesced through a per-doc queue with at most one in-flight pass and one pending. Flashcards, quizzes, and Feynman fire `scheduleEvaluation(docId)` when a session-worth of evidence lands (a deck closes, a quiz ends, a Feynman session wraps) and return immediately. Chat is chatty by definition, so it is deliberately **not** evaluated per reply: the student chats freely and the client fires a single pass when they leave the Chat tab (`POST /api/kg/[docId]/evaluate`). A pass that finds no new interactions since the last one returns without spending a call. The client polls `/api/kg/[docId]/state` (which exposes the live `evaluating` flag), accelerating to 2.5 s while the agent is working and slowing to 6 s when idle. The badge in the top tab bar reads "Building graph", "Evaluating", "No evaluations yet", or "Synced 12 s ago" depending on what the queue is doing.

An evaluator pass that fails — including on an account-level Codex error — simply stops; evaluation is best-effort background scoring, so the next genuine tool interaction schedules a fresh pass and the graph catches up then. We deliberately do **not** auto-retry on a timer (see *Resilience to Codex outages* for why that pattern was removed).

The four study tools and the work-context journal

The four tools are deliberately small and deliberately different. Each provides a distinct evidence type.

- **Chat.** Multi-turn, multi-thread, scoped to one document. The first turn opens a native Codex thread seeded with the knowledge-graph node list and the full document text; later turns resume that thread (by stored `threadId`) and send only the new message, so the document is transmitted once per conversation instead of re-injected on every reply. The student can chat freely across as many turns as they like; a single KG re-evaluation runs when they leave the Chat tab.
- **Flashcards.** Open-recall under self-grade. The student picks a topic (or "all"), Codex generates a 4–10 card deck, the student optionally types their answer, reveals, and self-grades 1–4 (Again / Hard / Good / Easy, the FSRS convention). Ratings are recorded per card; closing a deck triggers an evaluator pass.
- **Quizzes.** Forced-choice discrimination. Codex generates a 4–8 question multiple-choice quiz; each item carries one correct option and three plausible distractors picked to expose the confusion a student would actually trip on. The server **shuffles the options** at generation time with `crypto.randomInt` -driven Fisher–Yates so the agent's positional bias (the model tends to put the right answer at index 0) does not leak to the UI. The student picks, gets immediate feedback with a one-sentence explanation, and the quiz ends with a score summary.
- **Feynman.** The agent plays a curious eight-year-old who asks 3 to 4 short, pointed questions. The student is forced into the role of the teacher. After the last turn a separate summary call writes a 3–6-sentence honest read of where the explanation held and where it broke down. The session is bounded so the data stays usable for the evaluator and the student does not drift.

Behind all four sits one artifact: the **work-context JSON**, one file per doc on the server, append-only by convention. Every chat message, every card rating, every quiz answer, every Feynman turn lands here with a UTC timestamp. It is the file the evaluator reads, the file the student can download from the right-pane menu, and by design the only thing the system needs to remember about a study session. Backwards-compatible loading (`loadWorkContext`) back-fills any array that did not exist when the doc's journal was first written, so quizzes added in v1.1.0 work cleanly against pre-quiz journals from v1.0.0; it back-fills new optional fields the same way — per-interaction timestamps (which the incremental evaluator filters on) and the chat's Codex `threadId` — so journals written before v1.2.0 evaluate and resume without a migration step.

Persistent state and the types-split pattern

Filesystem-backed under one OS-native data directory per user, resolved once in `lib/paths.ts` .

OS	PATH
macOS	~/Library/Application Support/get-it/
Windows	%APPDATA%\get-it\
Linux	~/.local/share/get-it/

Or whatever the Electron main pinned via the `GETIT_DATA_DIR` environment variable. Layout:

```
docs.json           # top-level catalog
docs/<docId>/source.pdf # original bytes
docs/<docId>/meta.json # { id, filename, uploadedAt, numPages, lastOpenedAt }
docs/<docId>/extracted.json # cached pdfjs-dist output (text + bboxes per page)
docs/<docId>/tags.json  # server-owned visualizer tags + viz specs
docs/<docId>/workctx.json # the journal: chats / flashcards / quizzes / feynman
docs/<docId>/kg.json    # the knowledge graph + per-node scores
codex-scratch/        # Codex CLI's per-call working dir
logs/                 # embedded server stderr
settings.json         # auto-generate, max-repair-attempts
```

Cheap, recoverable, OS-agnostic, and a clear seam to lift to a hosted backend if we ever want to.

Types-split pattern. Modules are split into pure-TS `*-types.ts` files (no `node:fs` imports) and storage helpers in `*.ts` that do touch the filesystem. Next.js bundles a transitively-imported module into the client when *any* type from it is referenced, including a bare `import type {}`. Splitting types into a node-free file is the only way to keep `lib/kg.ts` and `lib/work-context.ts` server-only without poisoning the browser bundle. The comments in those files say so explicitly.

Settings. `lib/config.ts` reads env defaults for the two runtime-mutable settings (`NEXT_PUBLIC_AUTO_GENERATE_VIZ`, which defaults to off so visualizations are generated lazily on click, and `NEXT_PUBLIC_MAX_VIZ_GEN_RETRIES`) and persists overrides to `<DATA_DIR>/settings.json`. The dynamic localhost port the packaged app binds to changes on every launch, so anything cookie- or localStorage-scoped to the origin would forget the user's choice; a plain file in the user-data dir is the only thing that survives a restart. A change broadcasts a `getit:settings` window event so other pages on the same renderer react without polling.

Resilience to Codex outages

Every Codex call funnels through `runJson` in `lib/codex.ts`. That helper classifies failures into five kinds and writes the latest one into a process-local **health mailbox**.

KIND	TRIGGER	UI BEHAVIOUR
<code>auth_lost</code>	401 / token revoked / "sign in" message	Banner + "Re-connect" button re-opens the desktop setup wizard
<code>rate_limit</code>	429 / "try again in N" / 5-hour / weekly window phrases	Banner with a live countdown to <code>retryAt</code> (a fallback cooldown when the message carries no deadline); auto-clears when the window passes
<code>binary_missing</code>	Codex binary not found at the resolved path	Banner + button to re-install via the wizard
<code>model_unsupported</code>	"model is not supported" / the pinned model retired server-side	Banner telling the user to download the latest Get It.
<code>generic</code>	Anything else	Banner with the raw message

The in-app banner. The renderer polls `/api/codex/health` (fast cadence while there is an active problem, slow cadence otherwise) and renders `components/CodexHealthBanner.tsx`. The countdown updates locally so the banner stays smooth between polls.

Fail fast, retry by hand — never auto-loop. Earlier versions auto-resumed background work on a `setTimeout` keyed off `retryAt`. That had a sharp edge: a ChatGPT-account quota message often carries no parseable deadline, so `retryAt` was undefined, the backoff gate (which keyed on it) was never taken, and the viz queue re-picked the same still-`generating` tags and re-hit the wall as fast as calls completed — a tight loop that re-opened the banner the instant the user dismissed it. The fix is two-part: every rate-limit now gets a concrete deadline (parsed, or a

fallback cooldown) so the short-circuit always fires, and more fundamentally **no background job auto-retries on a Codex error**. Instead each surface stops cleanly and offers a manual retry:

- **Viz queue + detection** stop on any account-level error and drop the `generating` spinner from every still-pending tag, returning them to an idle, click-to-retry state (plus a Retry button in the Visualizer footer). A generic per-concept failure marks only that tag and the queue moves on.
- **KG build** moves to an `error` state with a Retry button; **KG evaluation** just stops and is re-triggered by the next interaction.
- **Request/response tools** (chat / flashcards / quizzes / Feynman) return a friendly `{ kind, message }` instead of an opaque 500, and the view shows it inline next to a Retry control. Chat's send is **atomic** — the user message and the reply are committed together, only on success — so a retry never duplicates the turn; Feynman's "explain" turn is atomic the same way and rolls back its optimistic state on failure.

The one retry loop that stays is the visualizer's **code-repair** loop (a sandbox runtime error feeds the broken code back to Codex for a corrected spec, bounded by `max-repair-attempts`). That is a content-level fix for the model's own output, not a backend-outage retry, and it is unchanged.

`runJson` also short-circuits future Codex calls while a rate-limit window is still active. A chatty UI cannot burn a hundred wasted calls hoping the next one succeeds.

Bring-your-own-account as an architectural choice

The decision to drive every agent through the **user's own ChatGPT login over the official Codex CLI** is the choice that shapes the whole product. It is not cost-cutting and not a missing feature; it is a deliberate boundary.

There is no server-side OpenAI key, no shared pool of credits, and no app-side metering of model usage. The Electron shell bundles the Codex CLI binary per platform/arch. The first-launch wizard spawns `codex login` so the user authenticates against OpenAI directly. Every subsequent `codex exec` call runs against that account at whatever tier the user pays for. The app sees the same auth state Codex sees: a successful login, a rate-limit window, an expired token. Nothing more.

Three properties follow.

1. **No second subscription, ever.** Other AI-study tools layer a marked-up fee on top of an API key the vendor holds. Get It. cannot do that, because it never holds the key in the first place. ChatGPT Plus is the practical floor for sustained study sessions; the free tier signs in but its Codex allowance is intentionally small. Higher tiers give more headroom in the exact same flow.
2. **No data resale and no transit-stage intermediary.** Because we never proxy the model traffic through our infrastructure, there is no Get It. infrastructure for that traffic to flow through. Work-context journals, knowledge graphs, and per-doc folders all live under the user-data directory on local disk. There is no document upload step and no cloud sync. The single exception is an anonymous open/update ping (a random install id + app version + OS, no document content or PII, opt-out via `GETIT_DISABLE_ANALYTICS=1`) that powers aggregate user counts on the marketing site. "Download your data" is a one-click affordance, but the more honest framing is that there is nothing else to download.
3. **The transport is replaceable.** Codex CLI is one of several ways the app could speak to a model. We ship it today because it has the best ergonomics around per-tier login, its bundled binary is small, the official SDK gives us schema-typed responses without DIY enforcement, and it is the only path through which a ChatGPT Plus account can drive a developer-facing CLI without an extra API-key purchase. If a comparable bring-your-own-account transport for another provider appears, `runJson` is the single touchpoint that needs to change.

The same property protects the project legally. **Get It. is not affiliated with OpenAI, not endorsed by OpenAI, not sponsored by OpenAI**, and not a derivative work of any closed-source OpenAI software; it is an independent application that interoperates with the publicly released Codex CLI and uses the end user's own credentials. The student's use of OpenAI's models through Get It. is governed by OpenAI's own Terms of Use, Usage Policies, and Privacy Policy. Those documents are authoritative.

Desktop packaging

The Electron shell is the boring kind of shell: it does as little as possible.

`electron/main.js` acquires a single-instance lock, normalises the user-data directory to `get-it` (overriding Electron's default `Application Support/Get It` so the path matches the pure-Next dev default), runs the setup wizard, spawns the Next.js standalone server as a child Node process on a free localhost port, and points one Chromium `BrowserWindow` at `http://127.0.0.1:<port>`. There is no native menu reinvention, no custom IPC for application data, and no second renderer. The UI is the unchanged Next.js app.

We chose Electron over Tauri because we wanted a guaranteed Chromium runtime on every supported OS: Three.js, KaTeX, the new `Function(...)` LLM sandbox, and pdf.js fonts all behave identically on every machine the user can install on.

`electron/setup.js` owns the Codex life-cycle. The Codex CLI binary ships *inside* the app: it is a Rust binary packaged as an npm optional dependency (`@openai/codex-<platform>-<arch>`) that the SDK locates via `createRequire`. At build time `scripts/electron-prepare.mjs` fetches the correct platform tarball from the npm registry (so a cross-arch build from an Apple Silicon Mac can still produce a usable Windows installer) and stages it under `electron/codex-bin/<triple>/codex/codex(.exe)`. At runtime the setup module resolves that path first; if missing or out of date, an "Install Codex CLI" button downloads it on demand into the user-data dir. The OAuth sign-in is run by spawning `codex login` and capturing the success line from stdout. The wizard is a stand-alone `BrowserWindow` loaded from a plain `file:///` page with a minimal context-isolated preload bridge.

Two boot guards worth knowing about.

- The `window-all-closed` handler **does not auto-quit** while a `bootstrapping` flag is true. Without that flag, dismissing the update modal or the wizard (which are both their own `BrowserWindow`) becomes the *last* open window and the implicit auto-quit fires before `whenReady` can reach `createMainWindow()`. The flag flips to false the instant the main window opens.
- `ELECTRON_RUN_AS_NODE=1` is unset at boot. If that env var leaks in, Electron loads as plain Node and `app` is undefined, which manifests as the cryptic `Cannot read properties of undefined (reading 'requestSingleInstanceLock')`. We catch that case and unset before any API touches `app`.

Build and release pipeline

Multi-target builds run from `scripts/build-electron.mjs`. Locally:

```
node scripts/build-electron.mjs --target=mac-arm64 # or mac-x64 / win-x64 / linux-x64 / --all
```

CI: pushing a `v*.*.*` tag to `main` triggers `.github/workflows/release.yml`. The workflow:

1. Rewrites `package.json#version` from the pushed tag so the same number flows into Info.plist / NSIS metadata, into `NEXT_PUBLIC_APP_VERSION` for the in-app version chip, and into the asset filenames.
2. Builds each target on a native runner: macOS Apple Silicon and macOS Intel both run on `macos-latest`, the latter cross-building via `electron-builder --mac --x64` because GitHub's Intel runners (`macos-13`) are being deprecated and queue times are unreliable. Windows builds on `windows-latest`, and Linux x64 builds a portable `.AppImage` on `ubuntu-latest` (electron-builder bundles `appimagetool`; the macOS signing steps are gated to the mac targets, so Linux skips them). There are no native modules in the bundle (the standalone server is pure JS, the Codex binary — including the static-musl Linux build — is fetched per target by `electron-prepare.mjs`) so cross-arch is clean.
3. Uploads each artefact to a workflow artifact.
4. A final `publish` job collects them and creates the GitHub Release tied to the tag.

macOS builds are **signed with a paid Apple Developer ID Application certificate and notarized by Apple** in the same `scripts/build-electron.mjs` invocation that produces the `.dmg`. The pipeline auto-detects what the host has:

- **developer-id** — a "Developer ID Application" identity is in the keychain *and* the App Store Connect API key trio is exported (`APPLE_API_KEY` path to the `.p8`, `APPLE_API_KEY_ID`, `APPLE_API_ISSUER`). `build-electron.mjs` adds `--config.mac.hardenedRuntime=true` `--config.mac.notarize=true` to electron-builder, which signs every Mach-O with the cert, ships the bundle to Apple's notary service, and staples the returned ticket onto the `.app` *inside* the `.dmg`. Gatekeeper opens the download with no prompt because the stapled ticket is consulted before the network is.

- **developer-id-no-notary** — cert is present but no notary credentials. We still sign (and keep Hardened Runtime on so the build is notarizable later) but skip the notary call. Useful for one-off local checks before secrets land in CI.
- **ad-hoc** — neither the cert nor the env-var pair is present (or `CSC_IDENTITY_AUTO_DISCOVERY=false` was set explicitly to force the legacy path). The `afterSign` hook (`scripts/electron-after-sign.cjs`) runs `codesign --force --deep --sign -` over the staged `.app` so the Apple Silicon kernel's mandatory-signature check still passes; without that step M-series Macs reject the bundle outright as "damaged" rather than showing the bypassable Gatekeeper prompt. First launch needs a one-time System Settings → Privacy & Security → Open Anyway dance.

The `afterSign` hook reads `process.env.GETIT_MAC_SIGNING_MODE` to decide which branch ran upstream. In the two `developer-id*` modes the hook only re-verifies the existing signature — overwriting a Developer ID signature with an ad-hoc one would break notarization. In `ad-hoc` mode the hook performs the codesign pass.

CI plumbing lives in `.github/workflows/release.yml`. The macOS matrix jobs decode two secrets into runner-temp files (`MAC_DEVELOPER_ID_CERT_BASE64` → a one-shot keychain via `security create-keychain` + `security import` + `security set-key-partition-list`; `APPLE_API_KEY_BASE64` → a `.p8` at a path exported via `$GITHUB_ENV`) before `build-electron.mjs` runs, then the same detection picks the `developer-id` branch automatically. The keychain and key file are scoped to the runner and disappear with the VM.

Windows builds are not signed: SmartScreen reputation is per-certificate and the project doesn't currently pay for a Windows code-signing cert. The first launch on Windows still shows the SmartScreen prompt; click **More info → Run anyway**.

Auto-update

On boot, before the wizard, `electron/updater.js` calls the GitHub Releases API for `beltramatti/get-it`, semver-compares its tag against `app.getVersion()` (the value the CI step pinned), and picks the asset whose filename matches the running platform and arch. When a newer version exists, a polished `BrowserWindow` shows the release notes and an "Update now" button. Clicking it downloads the asset with a live progress bar, hands the file to `shell.openPath` (Finder mounts the `.dmg`, Windows runs the NSIS installer, Linux surfaces the `.AppImage` to the file manager), and quits so the installer can replace the app on disk.

The user's library, work-context journals, knowledge graphs and settings all live outside the app bundle, so an in-place install never touches them. Network failures, 404s when no release is published yet, and assets missing for the running platform all silently bypass; the rest of startup proceeds unaffected.

Origin and trajectory

Get It. was built in 24 hours at **GDG AI Hack 2026, Milan**, for the **Braynr** challenge. Hackathon team:

- Mattia Beltrami (Politecnico di Milano)
- Matteo Impieri (Politecnico di Milano)
- Filippo Difronzo (Politecnico di Milano)
- Luca Feggi (Università di Padova)

The hackathon submission lived at commit `277ec43` and contained the core architecture this writeup describes: the visualizer pipeline with all five renderer types, the knowledge-graph build agent, the four-axis evaluator, the chat / flashcards / Feynman tools, and the work-context journal. Two design decisions that look obvious in hindsight come straight from the time constraint:

- **new Function for the LLM-emitted JS** was the only sandbox we could plausibly ship in 24 hours. We documented it as a defense against LLM mistakes rather than adversarial input; the boundary has held up because the bring-your-own-account model means the user is running their own Codex calls against their own PDFs.
- **Filesystem-only persistence**. Spinning up a database under a hackathon clock would have eaten the time we needed for the evaluator. The JSON-files-under-a-data-dir layout was a deadline call. It then turned out to be the right call once we added the desktop shell: the same files are now what the auto-update flow preserves across version bumps, and the same files are what the user downloads in a click.

Everything beyond `277ec43` is post-hackathon polish that turned the demo into a shipping product. Roughly chronological:

- **Server-side jobs runner.** Detection and per-tag viz generation moved from renderer loops into singleton-per-doc jobs inside the Next process. The viewer became a poll-and-display consumer. Multi-doc parallel progress and reopen-where-you-left-off both fell out for free.
- **Persistent Library** with `lastOpenedAt`, tag-progress and KG-status badges that poll the same job source as the viewer.
- **Desktop shell.** Electron main, embedded server, free-port spawn, single-instance lock. The renderer is byte-identical to the hackathon Next app.
- **First-launch setup wizard.** Bundled Codex binary, OAuth sign-in capture, re-entry on `auth_lost`.
- **Auto-update.** GitHub Releases poll on boot, in-app installer flow, no data loss across version bumps.
- **Codex error classifier + health mailbox.** The four-category banner with retry-deadline countdown, plus the evaluator queue's automatic resume.
- **Quizzes tool** (v1.1.0). The fourth study surface, with `crypto.randomInt`-driven option shuffle so the agent's positional bias does not leak.
- **Cross-arch CI.** Both macOS targets now build on `macos-latest`; the Intel slice cross-compiles.
- **Bring-your-own-account messaging.** The Notice, the writeup section above, the in-app wizard copy: all aligned so the legal posture and the product positioning are the same sentence.
- **Long-document support** (v1.2.0). The push that makes a 100-page PDF usable without exploding the user's Codex usage, at unchanged output quality. A model-free upload-quality gate rejects scanned / image-dominant / over-long files before any agent runs. Concept detection batches up to five pages per call. Chat moved onto a native Codex thread, so the document is sent once and follow-ups only carry the new message. KG evaluation became incremental (baseline scores plus only the new interactions) and chat now batches a single pass per visit instead of one per reply. Per-call character caps were removed across every prompt so the agents reason over whole sections rather than truncated fragments, and visualization generation defaults to lazy/on-click.
- **Signed and notarized macOS** (v1.2.1). Developer ID signing plus Apple notarization end to end, so a fresh download opens with no Gatekeeper prompt (see *Desktop packaging*).
- **Reliability and reach** (this release). The model is now pinned explicitly (`gpt-5.5`) so a retired binary default can't 400 users out of every generative feature, with a dedicated `model_unsupported` banner. Every background job stops cleanly on a Codex error and offers a manual retry instead of auto-looping — the fix for a rate-limit retry loop that could re-fire the banner endlessly. **Linux x64** joined macOS and Windows as a first-class AppImage target. An anonymous open/update ping powers real Total / Daily / Weekly / Monthly user counts on the marketing dashboard, cleanly separating genuine installs from in-app updates.
- **Open source, in the open.** The project is Apache-2.0 and actively seeks contributors; a [CONTRIBUTING.md](#) lays out the vision and a Discord community is where the work is coordinated.

The hackathon clock is no longer a load-bearing constraint, but the product it forced us into has not moved.

What's not here yet

A few choices are deliberately deferred.

Vocal Feynman. The same agent loop and the same end-of-session summary, with a streaming TTS layer over the child voice. The text variant ships today because typed transcripts are strictly better evaluator inputs (no transcription error, no per-token cost, full searchability). The data shape does not change.

A hosted multi-user backend. Out of scope by design. Get It. runs locally against the user's own Codex login, against their own PDFs, on their own machine. The Braynr policy band (source-grounded only, local-first, tiered access) we get for free at this scale.

Windows code signing. Windows builds are still unsigned: SmartScreen reputation is per-certificate and Microsoft's path to a Gatekeeper-equivalent zero-warning download (Azure Trusted Signing) requires a paid Azure subscription the project doesn't carry. The macOS notarization story is already in place (see *Desktop packaging* above); Windows is the remaining funding decision, not a missing piece of the architecture.

Notice and license

Get It. is an independent project. It is not affiliated with, endorsed by, sponsored by, or otherwise associated with OpenAI. The app uses the official open-source [Codex CLI](#) as the transport between the local app and OpenAI's

models, signed in with the end user's own ChatGPT or OpenAI API account. "OpenAI", "ChatGPT", and "Codex" are trademarks of their respective owner; we use the names only to describe what Get It. interoperates with.

Your use of OpenAI's models through Get It. is subject to OpenAI's own [Terms of Use](#), [Usage Policies](#), and [Privacy Policy](#), and to the Codex CLI's [own license and release notes](#). Those documents are authoritative for what the model service permits and how data is handled on OpenAI's side.

Source code is licensed under the **Apache License, Version 2.0**. See [LICENSE](#) .